



Logo Fast and Mudry - AI generated

RAPPORT FAST & MUDRY

Projet de POO

Axel Schneider

Helder Ribeiro

11 juin 2026

Table des matières

1 – Introduction	3
2 – Architecture du projet	3
3 – Génération de la route et de la carte	5
3.1 – La ligne centrale	5
3.2 – De la géométrie à l'image	6
3.3 – Les biomes et les objets	6
4 – La machine à états	6
4.1 – Les états et les évènements	6
4.2 – Le fonctionnement	7
5 – Musique et gestion du son	7
5.1 – La musique	7
5.2 – Les bruitages	8
6 – Le rendu pseudo-3D avec le Mode 7	8
6.1 – Fonctionnement du rendu Mode 7 avec le GPU	8
6.2 – Affichage des objets et calcul de profondeur	10
7 – Défi et problématique	11
7.1 – Rendu graphique	11
8 – Conclusion	11
9 – Annexes	12
9.1 – Liens et références	12

Note : Le rapport a été générée avec l'assistance d'une intelligence artificielle. Le contenu a été revu et validé par l'auteur avant son intégration au document final.

1 – Introduction

Dans le cadre du projet de programmation orientée objet, nous avons développé Fast & Mudry, un jeu de course en 2D écrit en Scala avec la bibliothèque gdx2d. Le principe est simple : le joueur contrôle une voiture qui doit traverser un circuit généré aléatoirement, en évitant les obstacles et en faisant attention à ne pas casser son véhicule en chemin.

Le développement ne s'est pas fait en ligne droite. La plus grosse évolution du projet a été le changement complet du système de rendu : les premières versions utilisaient une perspective calculée à la main dans le code, et nous l'avons remplacé par un rendu de type Mode 7 basé sur un shader qui tourne sur le GPU. Ce changement nous a obligé à réorganiser une partie du moteur de rendu, mais la logique du jeu elle-même a pu être conservée presque telle quelle.

Ce rapport présente d'abord l'architecture générale du projet, puis la génération de la route et de la carte, la machine à états qui gère le déroulement du jeu, la gestion de la musique et des sons, le fonctionnement du rendu pseudo-3D avec le Mode 7 et l'affichage des objets, et enfin les défis qu'on a rencontrés sur le rendu graphique.

2 – Architecture du projet

Le projet est organisé autour de trois grands axes : la logique de jeu, la gestion des écrans et le rendu graphique. Cette séparation n'était pas parfaite dès le début, mais nous avons essayé de la garder le plus possible pour que le code reste lisible et qu'on puisse travailler à deux sans se marcher dessus.

Les trois schémas ci-dessous montrent le système sous trois angles complémentaires : le flux d'application, le flux de données et le flux de rendu.

Le premier schéma montre le parcours dans l'application. Le programme commence par initialiser la fenêtre principale, puis enchaîne les écrans les uns après les autres : le menu, le chargement, la partie, le garage, l'écran de fin... L'application ne se résume donc pas à un programme linéaire : elle fonctionne comme une machine à états où chaque écran représente une étape du jeu (on y revient plus en détail dans la partie dédiée).

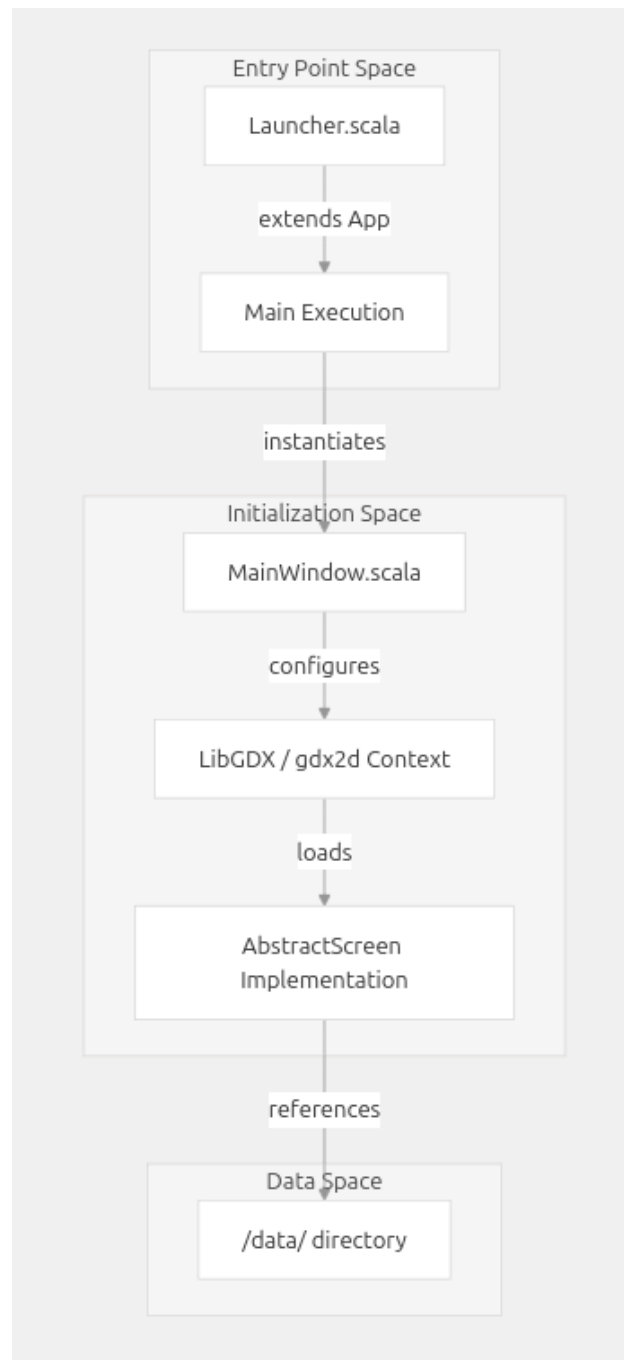


Figure 1 - Flux d'application du projet.

Le deuxième schéma se concentre sur les données. Tout tourne autour du monde du jeu : les entrées du joueur, les images, les sons et la progression alimentent un état global partagé. C'est à partir de cet état que la logique met à jour la position de la voiture, la progression du joueur, les événements du circuit et les objets interactifs. Ce qu'on voulait montrer ici, c'est que le cœur du projet n'est pas dans l'affichage, mais bien dans la gestion cohérente des informations du jeu.

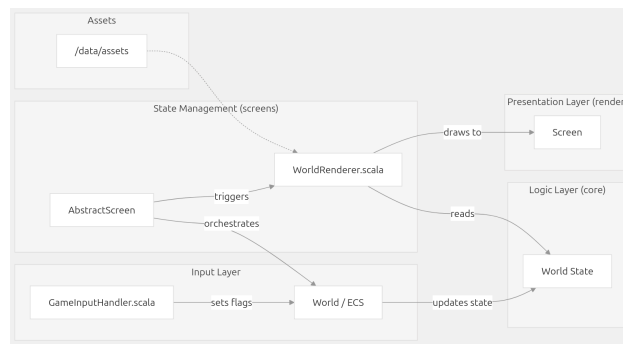


Figure 2 - Flux de données du projet.

Le troisième schéma explique comment cet état est ensuite transformé en image. Le module de rendu lit les informations du monde et les passe aux différents sous-systèmes de dessin : le fond, la piste, les objets, l'interface et le HUD. Pour la piste, le rendu repose sur une projection pseudo-3D réalisée par un shader Mode 7. L'affichage n'est donc pas une simple copie de l'état du jeu, c'est plutôt une interprétation visuelle de celui-ci.

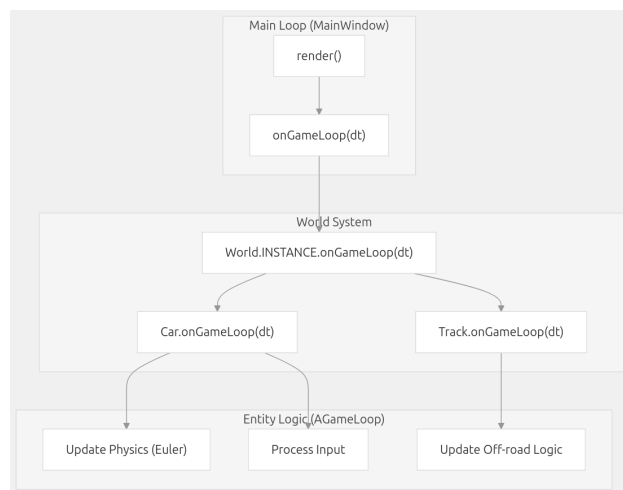


Figure 3 - Flux de rendu du projet.

Au final, ces trois vues se complètent bien : l'application gère les états, le cœur du jeu met à jour les données, et le moteur graphique transforme ces données en image. C'est cette séparation entre logique, données et rendu qui rend le projet modulaire et plutôt facile à faire évoluer.

3 – Génération de la route et de la carte

Un des objectifs du projet était que chaque partie soit un peu différente. Plutôt que de dessiner les circuits à la main, nous avons choisi de générer la route de manière procédurale au début de chaque journée de jeu. À chaque chargement, le joueur découvre donc un circuit qu'il n'a jamais vu.

3.1 – La ligne centrale

Tout part de la classe `TrackGeometry`, qui construit la ligne centrale de la route. Elle reçoit un point de départ, un point d'arrivée et un nombre de points de contrôle (20 dans notre cas). Les points sont d'abord répartis régulièrement entre le départ et l'arrivée, puis chacun reçoit un décalage aléatoire vertical et horizontal, sauf le premier et le dernier qui restent fixes pour garder un départ et une arrivée propre. À ce stade, la route ressemble à une suite de segments anguleux, ce qui n'est pas très agréable à conduire. Les points sont donc ensuite lissés avec une spline de Catmull-Rom, qui interpole 30 sous-points entre chaque paire de points de contrôle. Le résultat est une courbe fluide avec des virages naturels.

```
1 val yNoise = ((Math.random - 0.5) * 2.0 * randomHeight).toFloat
```

```

2  val h: Float = start.y + height * i + yNoise
3  val xNoise = if (i == 0 || i == nPoints - 1) 0f
4              else ((Math.random - 0.5) * 2.0 * width * 0.4).toFloat
5  val p: Vector2 = new Vector2(start.x + width * i + xNoise, h)

```

Listing 1 - Génération des points de contrôle aléatoires de la route

Cette ligne centrale sert ensuite de référence pour tout le reste. La géométrie expose une méthode qui calcule la distance entre un point quelconque et la ligne, ce qui permet de savoir si la voiture est sur la route ou non. Comme ce calcul est appelé énormément de fois, chaque segment possède une boîte englobante qui permet d'ignorer rapidement les segments trop éloignés. La ligne d'arrivée est simplement définie comme un cercle autour du dernier point de la courbe.

3.2 – De la géométrie à l'image

La carte affichée à l'écran est en réalité une grande texture générée par TrackTexture. L'objet parcourt chaque pixel de l'image et calcule sa distance à la ligne centrale. En fonction de cette distance, le pixel devient soit la ligne du milieu de la route, soit la route elle-même, soit le bas-côté, soit le terrain hors-piste. Les pixels proches du point d'arrivée sont dessinés en noir pour matérialiser la ligne d'arrivée.

Pour éviter d'avoir des surfaces complètement uniformes, les couleurs ne sont pas fixes. Chaque biome fournit une couleur claire et une couleur foncée pour chaque type de surface, et un bruit de Perlin vient mélanger les deux. Ça donne des textures avec des couleurs un peu différentes d'un pixel à l'autre, ce qui rend le sol beaucoup plus vivant. Le même principe est utilisé pour faire varier légèrement la largeur du bas-côté, comme ça le bord de la route n'est pas une ligne parfaite.

3.3 – Les biomes et les objets

Chaque journée du jeu possède son propre biome : la forêt, le désert et la neige. Le biome définit les couleurs de la carte, la musique, le décor en arrière-plan et aussi la physique quand on sort de la route. C'est également lui qui place les objets sur la carte, comme les arbres, les rochers ou les cactus. Pour chaque objet, une position est tirée aléatoirement sur la carte, et on recommence le tirage tant que la position ne convient pas, par exemple pour éviter qu'un arbre se retrouve au milieu de la route. Le bâtiment de la HES, lui, est toujours posé exactement sur le point d'arrivée.

Comme la génération de la texture est assez lourde (chaque pixel demande un calcul de distance), elle est lancée dans un thread séparé pendant que l'écran de chargement affiche une petite animation. Ça nous a permis d'éviter de bloquer la fenêtre du jeu pendant la génération. Une fois le calcul terminé, la texture est envoyée au GPU, la voiture est placée sur le premier point de la ligne centrale et orientée vers le deuxième, et la partie peut commencer.

4 – La machine à états

Le jeu est composé de plusieurs phases bien distinctes : le menu, le chargement, la course, les cinématiques, le quiz, le garage... Au début, les changements d'écrans étaient faits un peu à la main, mais on s'est vite rendu compte que ça devenait difficile de savoir qui avait le droit de passer où. Pour mettre de l'ordre, nous avons centralisé toute cette logique dans une machine à état.

4.1 – Les états et les événements

Chaque phase du jeu est représentée par un état, défini dans le trait scellé GameState. On y retrouve par exemple Menu, Loading, Playing, Quiz, Garage, Dead ou encore FinalCinematic. Certains états transportent en plus le jour courant de la partie (Day1, Day2 ou Day3), ce qui permet de savoir où le joueur en est dans sa progression sans avoir besoin d'une variable globale à côté.

Les transitions sont déclenchées par des événements, comme StartGame, MapLoaded, FinishLineCrossed ou CarBroke. L'idée centrale est que chaque état décide lui-même de sa transition : il implémente

une méthode `next` qui reçoit un évènement et retourne le prochain état. Si l'évènement ne le concerne pas, l'état se retourne simplement lui-même et rien ne se passe.

```

1  final case class Playing(day: Day) extends GameState {
2      def next(event: GameEvent): GameState = {
3          event match {
4              case FinishLineCrossed if day.next.isEmpty => FinalCinematic
5              case FinishLineCrossed => EndDayCinematic(day)
6              case CarBroke => Dead
7              case _ => this
8          }
9      }
10 }

```

Listing 2 - L'état Playing et ses transitions possibles

4.2 – Le fonctionnement

L'objet `GameStateMachine` garde en mémoire l'état courant. Quand un évènement arrive, il demande à l'état courant quel est le prochain état. Si l'état change, la machine active l'écran qui correspond via le gestionnaire d'écrans : l'état Menu affiche l'écran du menu, l'état Playing affiche l'écran de jeu, et ainsi de suite. Si l'état ne change pas, l'évènement est simplement ignoré. C'est un gros avantage : un évènement envoyé au mauvais moment ne peut pas casser le jeu ni provoquer des comportements bizarres.

Les évènements sont envoyés depuis un peu partout dans le code. Les boutons du menu envoient `StartGame` ou `OpenCarSelector`, l'écran de chargement envoie `MapLoaded` quand la carte est prête, la piste envoie `FinishLineCrossed` quand la voiture franchit la ligne d'arrivée ou `CarBroke` quand le véhicule est cassé, et les cinématiques envoient leur évènement de fin une fois leur durée écoulée. Aucun de ces endroits n'a besoin de connaître l'écran suivant, ils signalent juste ce qui vient de se passer.

Une partie complète suit donc un cycle assez naturel : depuis le menu, on passe au chargement puis à la course du premier jour. Quand la ligne d'arrivée est franchie, une cinématique de fin de journée se lance, suivie du quiz puis du garage où le joueur peut réparer sa voiture. Une cinématique de début de journée enchaîne ensuite sur le chargement du jour suivant. Au troisième jour, franchir la ligne d'arrivée déclenche la cinématique finale qui ramène au menu. Et si la voiture casse en route, peu importe le jour, on passe directement à l'écran de mort.

Ce découpage nous a beaucoup aidé sur la fin du projet : pour rajouter une nouvelle phase, comme le sélecteur de voiture, il suffisait de rajouter un état, ses évènements et son écran, sans toucher au restant de la logique.

5 – Musique et gestion du son

Un jeu de course sans son, ça ne donne pas grand chose. Nous avons donc ajouté de la musique et des bruitages, et toute cette partie est centralisée dans un seul objet, `AudioManager`. C'est lui qui possède les lecteurs de musique et les effets sonores, et le reste du code se contente de lui demander de jouer quelque chose, sans savoir comment ça marche derrière.

5.1 – La musique

Chaque morceau du jeu est représenté par un objet du trait scellé `MusicTrack` : il y en a un pour le menu, le quiz, le garage, les cinématiques, et un par biome pour la course (forêt, désert et neige). Quand un écran démarre, il demande simplement à `AudioManager` de jouer son morceau. Pendant la course, c'est le biome du jour qui fournit sa propre musique, donc l'ambiance sonore change en même temps que le décor.

La méthode `playTrack` s'occupe de la transition : elle arrête le morceau en cours et lance le nouveau en boucle. Si on lui demande le morceau qui est déjà en train de jouer, elle ne fait rien du tout, ce qui évite

que la musique soit relancer à zéro quand on revient sur le même écran. Les lecteurs sont gardés dans une map et ne sont créés qu'à la première utilisation d'un morceau, comme ça on ne charge pas tous les fichiers audio au lancement du jeu. Il y a aussi un petit cas particulier : la cinématique finale ne change pas de musique et garde celle du dernier biome, ce qu'on trouvait plus sympa pour la fin du jeu.

5.2 – Les bruitages

Le bruitage le plus important est celui du moteur. C'est un son joué en boucle dont la hauteur (le pitch) est recalculée à chaque frame en fonction de la vitesse de la voiture : plus on va vite, plus le moteur monte dans les tours. Le ratio entre la vitesse actuelle et la vitesse maximale est converti en un pitch compris entre une valeur basse et une valeur haute, ce qui donne l'impression d'accélérer alors qu'on joue toujours le même fichier son.

```

1 var ratio = Math.abs(speed) / maxSpeed
2 if (ratio > 1f) ratio = 1f
3
4 var pitch = AUDIO.SFX.ENGINE_LOW_PITCH + ratio * (AUDIO.SFX.ENGINE_TOP_PITCH -
5 AUDIO.SFX.ENGINE_LOW_PITCH)
6 if (pitch < 0.5f) pitch = 0.5f
7 if (pitch > 2f) pitch = 2f
8 engine.modifyPitch(pitch, engineId)

```

Listing 3 - Calcul du pitch du moteur en fonction de la vitesse

Notre petit plaisir du projet, c'est le backfire. Quand le joueur relâche les gaz à haute vitesse, le pot d'échappement lâche une série de petit pops, comme sur une vraie voiture de course. Le monde du jeu surveille la vitesse : si elle a dépassé un certain seuil puis commence à redescendre, il déclenche le backfire. AudioManager joue alors entre 4 et 5 pops, avec un délai et un pitch tirés aléatoirement pour chaque pop, comme ça la séquence n'est jamais exactement la même et ça sonne beaucoup moins artificiel.

Les derniers effets sont liés aux problèmes de la voiture : un son d'explosion quand un pneu lâche et un bruit de crash quand on percute un obstacle. Comme pour la musique, ces sons sont chargé seulement à la première utilisation. Et quand on ferme le jeu, la fenêtre principale demande à AudioManager de libérer toutes les ressources audio proprement.

6 – Le rendu pseudo-3D avec le Mode 7

6.1 – Fonctionnement du rendu Mode 7 avec le GPU

Dans l'architecture actuelle, c'est TrackRenderer qui est responsable de créer l'image affichée. Il joue un rôle d'intermédiaire : il récupère la texture du circuit, place une caméra virtuelle en fonction de la position du véhicule, envoie une série de paramètres au shader et lance le rendu. Cette séparation est pratique car la logique du jeu reste dans les classes métier, pendant que la partie graphique s'occupe juste de transformer ces informations en image.

Le fichier Mode7.scala ne dessine rien lui-même. Il sert surtout de configuration pour le shader : les noms des paramètres, les chemins vers le programme graphique et les valeurs par défaut. TrackRenderer transmet ensuite tout ça au shader : la résolution de l'écran, la position de la caméra, son angle, l'axe de rotation, la distance du plan de projection et le facteur d'échelle de la texture. L'avantage, c'est que la projection reste dans un module spécialisé, complètement indépendant de la physique de conduite ou de la gestion du jeu.

La position et l'orientation de la voiture influencent directement le rendu. À chaque image, TrackRenderer met à jour la caméra à partir des coordonnées du véhicule et de sa rotation, puis envoie ces valeurs au shader qui recalcule l'affichage du circuit pour chaque pixel. Du coup, quand la voiture tourne, la

texture du circuit est réinterprété sous un autre angle, ce qui donne l'illusion qu'on se déplace sur une vraie route en perspective. On peut donc voir le véhicule comme une entrée du rendu, au même titre que la taille de l'écran ou la résolution de la texture.

```

1 cameraPosition.y = World.INSTANCE.CAR.Coordinates.y
2 cameraPosition.x = World.INSTANCE.CAR.Coordinates.x
3 cameraAngle = World.INSTANCE.CAR.Rotation
4
5 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.POSITION, cameraPosition)
6 g.getShaderRenderer.setUniform(Mode7.Parameter.KEY.CAMERA.ANGLE, cameraAngle)

```

Listing 4 - Transmission des informations de déplacement au shader

Côté shader, le calcul part des coordonnées du fragment, c'est à dire du pixel à afficher à l'écran. Pour chaque pixel, le shader construit un rayon de vue dans un espace virtuel, applique une rotation selon l'angle de la caméra, puis retrouve la position correspondante dans la texture du circuit. En combinant la position de la caméra, la distance du plan de projection et le facteur de mise à l'échelle, on obtient au final une coordonnée de texture comprise entre 0 et 1.

$$\begin{aligned}
 r &= (x_s, y_s, d) \\
 r' &= R(a)r \\
 p &= c + (r'_x, r'_y) * \left(\frac{c_z}{r'_z} \right) \\
 t &= (p - o) * f
 \end{aligned}$$

auto 4 - Formulation générale de la projection du fragment

Cette écriture résume le calcul fait par le shader : un fragment, défini par ses coordonnées écran x_s et y_s , correspond d'abord à un rayon r dans un espace virtuel. Ce rayon est tourné selon l'angle a de la caméra, puis projeté sur un plan de référence pour obtenir une position p dans le repère de la carte. La coordonnée de texture t est enfin obtenue en appliquant une mise à l'échelle à partir de l'origine o et du facteur f . Si cette coordonnée tombe sur le circuit, la couleur de cette zone est lue dans la texture et affichée à l'écran. Au final c'est de la géométrie assez simple, mais appliqué à chaque pixel par le GPU.

```

1 vec3 pixelRay;
2 pixelRay.xz = (gl_FragCoord.xy / resolution.xy) * vec2(1.0, -1.0) + vec2(-0.5,
3 0.5);
4 pixelRay.y = screenPlanDistance;
5 pixelRay *= rotationMatrix(cameraAxis, cameraAngle);
6
7 vec2 hitPosition = cameraPosition.xy + pixelRay.xy * (cameraPosition.z /
8 pixelRay.z) + vec2(0.5, 0);
9 hitPosition = (hitPosition - mapOrigin) * renderingFactor;
10 gl_FragColor = texture2D(backbuffer, hitPosition);

```

Listing 5 - Projection du fragment vers une coordonnée de texture

Le GPU est parfait pour ce genre de traitement, parce qu'il peut faire les calculs sur des millions de pixels en parallèle. Contrairement au CPU qui traiterait les pixels un par un, le shader les traite tous en même temps avec la même logique de projection. Le processeur graphique s'occupe donc de la transformation géométrique et de la lecture de la texture, ce qui laisse le CPU se concentrer sur la logique du jeu.

Cette méthode est simple et efficace pour un jeu de course en 2D/2.5D, mais elle a ses limites : tout dépend de la résolution de la texture, du réglage de la caméra et de l'alignement entre la piste et les objets du décor. Il a aussi fallu régler le shader avec soin pour éviter les artefacts au bord du circuit, là où la projection devient instable. C'est un compromis qui nous convenait bien : un rendu fluide sans rajouter une complexité de calcul trop importante.

6.2 – Affichage des objets et calcul de profondeur

La même logique de projection est réutilisée pour afficher les objets du jeu, comme les obstacles ou les éléments du décor, par-dessus le circuit. Chaque item possède une position dans le monde, exprimée dans les coordonnées du circuit, et il faut le projeter dans l'espace de l'écran en fonction de la position de la caméra et de son orientation. Pour ça, on calcule d'abord un vecteur entre la position de l'objet et celle de la caméra, puis on fait tourner ce vecteur selon l'angle du véhicule. Le résultat permet de savoir si l'objet se trouve devant ou derrière la caméra virtuelle : si la composante de profondeur est négative ou trop petite, on ne le projete pas du tout, ce qui évite d'afficher des objets qui sont derrière nous ou hors du champ de vision.

Quand l'objet est visible, sa position à l'écran est obtenue par une projection perspective simple. La position horizontale vient du rapport entre la distance latérale de l'objet et sa profondeur, et la position verticale dépend à la fois de la hauteur de la caméra et de l'inclinaison du point de vue. L'échelle de l'objet est elle aussi calculée à partir de la profondeur, avec une relation de la forme suivante.

$$s = \frac{d_p}{p_y}$$

auto 5 - Échelle de l'objet calculée à partir de sa profondeur relative

Grâce à cette relation, les objets proches paraissent gros et les objets lointain tout petits, ce qui donne une impression de profondeur cohérente entre tous les éléments de la scène.

Dans le code, tout ce calcul est centralisé dans ItemsRenderer. La méthode de projection retourne les coordonnées d'affichage, un facteur d'échelle et une distance, puis les objets sont triés selon cette distance avant d'être dessinés. Les coordonnées à l'écran sont obtenues avec les deux relations suivantes.

$$x_e = w * \left(0.5 + \frac{d_p * p_x}{p_y}\right)$$

auto 6 - Coordonnée horizontale de projection sur l'écran

$$y_e = h * \left(0.5 + t - \frac{d_p * z_c}{p_y}\right)$$

auto 7 - Coordonnée verticale de projection sur l'écran

Ce tri par distance est important : sans lui, des éléments du décor ou des obstacles pouvaient se superposer n'importe comment et la scène devenait illisible.

```

1  val differentialVector = new Vector2(position.x, position.y)
2  differentialVector.sub(cameraPosition.x + 0.5f, cameraPosition.y)
3
4  val primeVector = new Vector2(
5      differentialVector.x * cosAlpha - differentialVector.y * sinAlpha,
6      differentialVector.x * sinAlpha + differentialVector.y * cosAlpha
7  )
8
9  if (primeVector.y ≤ 0.01f) return None
10
11 val screenX = screenW * (0.5f + (screenPlanDistance * primeVector.x) /
12 primeVector.y)
13 val screenY = screenH * (0.5f + pitch - (screenPlanDistance * cameraPosition.z) /
14 primeVector.y)
15
16 val scale = screenPlanDistance / primeVector.y

```

Listing 6 - Projection d'un item dans l'espace de l'écran

Pour résumer le flux de données : l'état du véhicule est transmis sous forme d'uniforms par TrackRenderer, le shader GPU échantillonne la texture du circuit, et l'image finale s'affiche à l'écran. Pour les items, il y a juste une projection en plus, qui transforme leur position dans le monde en coordonnée d'écran et en échelle visuelle.

7 – Défi et problématique

7.1 – Rendu graphique

Un des plus gros défis du projet a été le rendu de la piste. Les premières versions du jeu utilisaient un affichage de type arcade, avec une perspective simulée directement dans le code. C'était relativement rapide à mettre en place et ça donnait déjà une impression de route avec des virages, mais on a vite vu les limites de cette approche.

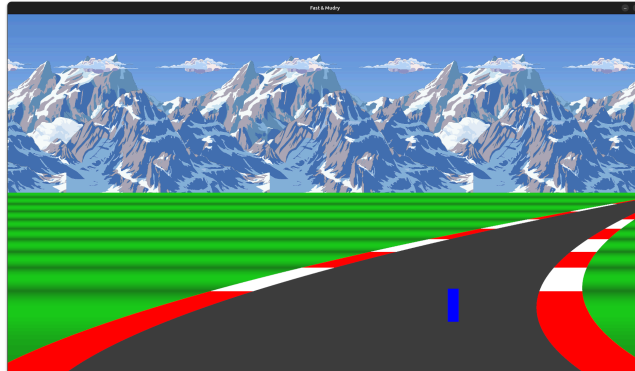


Figure 8 - Première version du rendu

En pratique, les virages étaient presque impossible à anticiper. La route manquait de profondeur et les changements de direction arrivaient d'un coup, sans prévenir. Pour le joueur c'était frustrant : on ne voyait que la portion de route juste devant la voiture, donc difficile de préparer une trajectoire ou d'éviter un obstacle qu'on découvre au dernier moment.

Pour corriger ça, nous avons repensé le système de rendu autour d'une approche de type Mode 7, la technique utilisée à l'époque par des jeux comme Mario Kart sur Super Nintendo. L'idée est de projeter une texture représentant le circuit de façon à créer une illusion de profondeur, tout en restant dans un environnement essentiellement 2D. Par rapport à l'ancien rendu, on voit une portion beaucoup plus grande du circuit devant le véhicule.



Figure 9 - Rendu avec mode 7

Ce changement a vraiment amélioré la lisibilité du jeu. Le joueur distingue maintenant les virages à venir, repère les obstacles situés plus loin sur la piste et peut voir certains éléments importants du décor, comme la HES qui marque la ligne d'arrivée. La conduite devient plus naturelle : on anticipe au lieu de simplement réagir à ce qui apparaît à l'écran.

8 – Conclusion

Ce projet nous a permis de construire un jeu complet, de l'écran de menu jusqu'à la cinématique finale, en passant par la génération procédurale de la carte, une machine à états, un rendu pseudo-3D sur le GPU et la gestion de la musique et des sons. Au départ, l'objectif était simplement de faire rouler une

voiture sur une route ; au final, on se retrouve avec trois biomes, un garage, un quiz et des pneus qui explosent. Au-delà du jeu lui-même, le projet nous a apporté beaucoup de connaissance sur la programmation orientée objet.

Sur le plan technique, la plus grande leçon du projet est probablement de ne pas avoir peur de jeter du code. Nous avons passé pas mal de temps sur le premier système de rendu, et le remplacer entièrement par le Mode 7 n'était pas une décision facile. Mais c'est ce changement qui a rendu le jeu réellement agréable à jouer.

Le travail à deux s'est bien passé, notamment grâce au découpage du code en modules et à l'utilisation de branches git avec des pull requests. Chacun pouvait avancer sur sa partie sans bloquer l'autre, et les conflits de merge sont restés rares.

Évidemment, tout n'est pas parfait. Certaines parties aurait mérité un peu plus de polissage, comme les réglages du shader qui restent un peu fragile au bord du circuit. Et si le projet devait continuer, on aimerait ajouter plus de journées avec de nouveaux biomes, enrichir la physique des collisions et pourquoi pas un mode deux joueurs. Mais dans l'ensemble, nous sommes plutôt fier du résultat : un jeu qui se lance, qui se joue, et qui donne envie de refaire une partie pour voir un circuit différent.

9 – Annexes

9.1 – Liens et références

Voici les différentes ressources qui nous ont aidé pendant le projet. La plupart sont aussi citées en commentaire directement dans le code, aux endroits concernés.

- [Programming Pseudo 3D Planes aka MODE7 \(javidx9\)](#) : la vidéo qui nous a servi de référence pour comprendre le fonctionnement du rendu Mode 7.
- [Code-It-Yourself! Retro Arcade Racing Game \(javidx9\)](#) : l'inspiration principale de la toute première version du rendu, de type arcade.
- [Let's make 16 games in C++ : Outrun \(FamTrinli\)](#) : un autre exemple de jeu de course en pseudo-3D qui nous a inspiré au début du projet.
- [Programming & Using Splines - Part#1 \(javidx9\)](#) : une vidéo sur les splines, utile pour comprendre le lissage de la ligne centrale de la route.
- [perlin-noise \(GitHub\)](#) : le dépôt d'où vient la majorité du code de génération de bruit de Perlin, utilisé pour les textures des biomes.
- [Scala Algorithms : StateMachine](#) : la structure sur laquelle est basé notre machine à états.
- [Stack Overflow : distance point-segment](#) : l'algorithme de calcul de distance entre un point et un segment, utilisé pour déterminer si la voiture est sur la route.
- [Documentation LibGDX : Scene2D](#) : la documentation utilisée pour la construction de l'interface utilisateur.